

Exercise 2: E-commerce Platform Search Function

Scenario

You are working on the search functionality of an e-commerce platform. The search needs to be optimized for fast performance.

1. Understand Asymptotic Notation

What is Big O Notation?

Big O notation is a mathematical representation used to measure the performance of an algorithm as the input size increases. It describes the worst-case growth rate of an algorithm and helps compare the efficiency of different algorithms.

Big O notation enables developers to choose algorithms that perform efficiently for large datasets by analyzing their time and space complexity.

Best, Average, and Worst-Case Scenarios

Best Case

- The minimum time required by an algorithm to complete its execution.
- Example: In Linear Search, the element is found at the first position.
- Time Complexity: $O(1)$

Average Case

- The expected execution time when the element is equally likely to be at any position.
- Example: In Linear Search, the element is found somewhere in the middle.
- Time Complexity: $O(n)$

Worst Case

- The maximum time required by an algorithm.
- Example: The element is found at the last position or is not present.
- Time Complexity:
 - Linear Search: $O(n)$
 - Binary Search: $O(\log n)$

2. Setup

A Java project named `ECommerceSearchExample` was created.

The project contains a code folder, which includes two separate Java files:

ECommerceSearchExample

└── code

├── Product.java

└── ECommerceSearchExample.java

- **Product.java** – Defines the Product class with attributes such as productId, productName, and category.
- **ECommerceSearchExample.java** – Implements Linear Search and Binary Search algorithms and demonstrates their execution.

3. Implementation

Product Class

The Product class represents each product available on the e-commerce platform.

Attributes

- productId
- productName
- category

Search Algorithms Implemented

Linear Search

- Stores products in an array.
- Traverses each element sequentially until the required product is found.
- Suitable for both sorted and unsorted data.

Binary Search

- Stores products in a sorted array.
- Repeatedly divides the search space into two halves.
- Requires the array to be sorted before searching.

4. Analysis

Time Complexity Comparison

Operation	Best Case	Average Case	Worst Case
Linear Search	$O(1)$	$O(n)$	$O(n)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$

Comparison of Linear Search and Binary Search

Linear Search

- Searches each element one by one.
- Works on both sorted and unsorted arrays.
- Suitable for small datasets.
- Slower for large collections because every element may need to be checked.

Binary Search

- Searches by repeatedly dividing the search space into two halves.
- Requires the data to be sorted.
- Much faster than Linear Search for large datasets.
- Efficient for applications where searching is performed frequently.

Which Algorithm is More Suitable?

For an e-commerce platform containing thousands or millions of products, Binary Search is more suitable because it significantly reduces the number of comparisons required to locate a product. Although the products must be sorted before performing Binary Search, its $O(\log n)$ time complexity makes it much more efficient than Linear Search for large datasets.

5. Conclusion

The E-commerce Platform Search Function was successfully implemented using both Linear Search and Binary Search algorithms. Products were stored in arrays, and searching was performed using both techniques. By comparing their time complexities, it is evident that Binary Search provides better performance for large, sorted datasets, making it the preferred choice for search-intensive e-commerce applications.